

Offline 7: Hashing
Deadline: August 19, 8:55 AM

1) Problem Specification

You need to implement a **HashTable** class for this assignment. The requirements are as follows:

- Data will be kept as (key, value) pairs in this hash table.
- You can *insert*, *delete*, and *find* data from this table.
- The minimum size N' of this table will be given as input, but you need to find the nearest prime N greater than or equal to N' as the actual **size of this table**.
- You need to use randomly generated words of lengths 5 to 10 as keys for this table. Therefore, you need to implement a random word generator. The lengths of the words will be randomly determined as well. The words do not have to be meaningful. Using only lowercase alphabetical letters will **suffice**. The order of the incoming words will be used as their value (please see the example in [Section 2](#) below). **If your word generator produces duplicate words, you must only keep one instance of each, discarding the others. Use your *find* function to ensure this.**
- Your hash table can have different collision resolution methods (see details in [Section 4](#)). **The collision resolution method of your hash table instance should be specifiable via the constructor.**

2) Example

Suppose your word generator has generated the following words:

puzzle
universe
brain
universe
offline

The corresponding (key, value) pairs will be:

(puzzle, 1)
(universe, 2)
(brain, 3)
(offline, 4)

Note that the second instance of the word “universe” has been discarded, and the value for “offline” has been set to 4 instead of 5.

3) Hash Function

You have to use two standard hash functions ($Hash1(k)$ and $Hash2(k)$) of your own, or from any good literature where you must try to avoid collisions as much as possible. We expect that 60% of the keys will have unique hash values (e.g., at least 60 unique hash values for 100 keys).

4) Collision Resolution

You need to implement the following three collision resolution methods:

1. **Separate chaining:** Place all the elements that hash to the same slot into a linked list. Slot j of the hash table contains a pointer to the head of a linked list which will store all the items that hash to the j -th slot. If there are no such elements, slot j contains *null*.

2. **Double hashing:** For double hashing, use the following hashing function:

$$doubleHash(k, i) = (Hash(k) + i \times auxHash(k)) \% N$$

Here, $Hash(k)$ is one of the hash functions described in [Section 3](#). Note that you have to use both the Hash functions mentioned in Section 3 for report generation (see [Section 5](#) for more details). You can use a simple hash function as the auxiliary hash function $auxHash(k)$. The initial probe goes to position $Table[Hash(k)]$, and successive probe positions are offset from previous positions by an amount $auxHash(k)$, modulo N .

3. **Custom probing:** For custom probing, use the following hash function:

$$customHash(k, i) = (Hash(k) + C_1 \times i \times auxHash(k) + C_2 \times i^2) \% N$$

Here C_1 and C_2 are two auxiliary constants of your choice. The other details are the same as the Double Hashing.

5) Report Generation

Generate 10000 unique words and insert them into the Hash Table. Using both hash functions ($Hash1(k)$ and $Hash2(k)$), list the number of collisions in a tabular format. Among these 10000 generated words, randomly select 1000 words and search each of these selected words in the hash table. Report the average number of probes (i.e., the number of times you access the hash table) required to search these words. These results should be reported for both hash functions. The report should be generated in the following tabular format. You also need to report the hash functions and the auxiliary hash functions that you have used.

Generate your report for $N' = 5000, 10000$, and 20000 .

Hash Table Size	Collision Resolution Method	Hash1		Hash2	
		# of Collisions	Average Probes	# of Collisions	Average Probes
	Chaining				
	Double Hashing				
	Linear Probing				

Note that, you may be asked to run your program during evaluation to show that the output of your program closely resembles the reported values (use a fixed seed to be able to reproduce your results exactly).

6) Submission Guideline

1. Create a directory with your 7-digit roll number as the name
2. Put only the source files and the report in the mentioned directory
3. Zip the directory in “.zip” format, rename it using your 7-digit roll number, and submit it on moodle within the mentioned deadline (**August 19, 8:55 AM**).

Failure to follow the above-mentioned instructions may result in up to 10% penalty.